

✓ Snack Market Gap Analysis

This notebook is the starting point for the analysis of the Open Food Facts snack dataset. It is configured for Google Colab and uses a subset of the first 500,000 rows for initial profiling and cleaning.

Objectives

- Load the first 500K rows of Open Food Facts.
- Profile key columns: `sugars_100g`, `proteins_100g`, `energy_100g`, and `categories_tags`.
- Define cleaning rules and prepare the dataset for category mapping and visualization.

✓ 1) Environment Setup

The following cell installs required packages if needed. In Colab, this can be run once per session.

```
import pandas as pd
import numpy as np
import plotly.express as px
from pathlib import Path

print('Libraries imported successfully')
```

Libraries imported successfully

✓ 2) Data Load

Load the first 500,000 rows from the Open Food Facts dataset. The dataset should be placed in the `data/` folder at the repository root as `openfoodfacts.zip`. This notebook uses a relative path (`../data/openfoodfacts.zip`) to ensure reproducibility.

```
# Public link to dataset
DATA_URL = "https://drive.google.com/uc?id=1AFsvaQER0rCHf3bnMCyYAEjjhBPdGQz"

# Dataset download destination
data_dir = Path('data')
data_dir.mkdir(parents=True, exist_ok=True)
data_path = data_dir / 'openfoodfacts.csv.gz'
print('Dataset path:', data_path.resolve())

if not data_path.exists():
    print('Dataset not found locally. Downloading from Google Drive...')
    import gdown

    try:
        gdown.download(DATA_URL, str(data_path), quiet=False)
        print('Download completed:', data_path)
    except Exception as exc:
        print('Dataset download failed:', exc)
        raise

import gzip

# Inspect the first few lines to confirm the delimiter
with gzip.open(data_path, 'rt', encoding='utf-8', errors='replace') as f:
    sample_lines = [next(f) for _ in range(5)]

print('Sample lines from file (first 5 lines):')
print(''.join(sample_lines))

# Open Food Facts export files are typically tab-separated
print('Loading dataset...')
df = pd.read_csv(
    data_path,
    compression='gzip',
    sep='\t',
```

```

nrows=500000,
low_memory=False
)
print(f'Loaded {len(df):,} rows from the dataset')
print('Dataset shape:', df.shape)
print('\nColumns:')
print(df.columns.tolist())
print('\nFirst few rows:')
display(df.head())

```

Dataset path: /content/data/openfoodfacts.csv.gz
Dataset not found locally. Downloading from Google Drive...
Downloading...
From (original): <https://drive.google.com/uc?id=1AFsvaQEROrCHf3bnMCyYAEjjhBPdGQOz>
From (redirected): <https://drive.google.com/uc?id=1AFsvaQEROrCHf3bnMCyYAEjjhBPdGQOz&confirm=t&uuid=9988bc42-0180-4>
To: /content/data/openfoodfacts.csv.gz
100%|██████████| 1.28G/1.28G [00:14<00:00, 85.5MB/s]
Download completed: data/openfoodfacts.csv.gz
Sample lines from file (first 5 lines):

code	url	creator	created_t	created_datetime	last_modified_t	last_modified_datetime	last_modified_by
000000000054	http://world-en.openfoodfacts.org/product/000000000054/limonade-artisanale-a-la-rose	kiliweb	1582569031	2020-02-24T18:30:31Z	1733085204	2024-12-01T20:30:31Z	kiliweb 1582569031
000000000063	http://world-en.openfoodfacts.org/product/000000000063/m-amp-m-white-fitpiggy	kiliweb	1673620307	2023-01-13T14:31:47Z	1750061386	2025-06-16T08:00:00Z	kiliweb 1673620307
000000000114	http://world-en.openfoodfacts.org/product/000000000114/chocolate-n3-jeff-de-bruges	kiliweb	1580066482	2020-01-26T19:21:22Z	1751035658	2025-06-27T14:40:00Z	kiliweb 1580066482
000000000431	http://world-en.openfoodfacts.org/product/000000000431/pate-de-fruits	kiliweb	1714301712	2024-04-28T10:55:12Z	1714301721	2024-04-28T10:55:12Z	kiliweb 1714301712

Loading dataset...
Loaded 500,000 rows from the dataset
Dataset shape: (500000, 211)

Columns:
['code', 'url', 'creator', 'created_t', 'created_datetime', 'last_modified_t', 'last_modified_datetime', 'last_modified_by']

First few rows:

	code	url	creator	created_t	created_datetime	last_modified_t	last_modified_datetime
0	54	http://world-en.openfoodfacts.org/product/000000000054/limonade-artisanale-a-la-rose	kiliweb	1582569031	2020-02-24T18:30:31Z	1733085204	2024-12-01T20:30:31Z
1	63	http://world-en.openfoodfacts.org/product/000000000063/m-amp-m-white-fitpiggy	kiliweb	1673620307	2023-01-13T14:31:47Z	1750061386	2025-06-16T08:00:00Z
2	114	http://world-en.openfoodfacts.org/product/000000000114/chocolate-n3-jeff-de-bruges	kiliweb	1580066482	2020-01-26T19:21:22Z	1751035658	2025-06-27T14:40:00Z
3	431	http://world-en.openfoodfacts.org/product/000000000431/pate-de-fruits	kiliweb	1714301712	2024-04-28T10:55:12Z	1714301721	2024-04-28T10:55:12Z
4	105	http://world-en.openfoodfacts.org/product/000000000105/le-pain-de-seigle-artisanal	kiliweb	1572117743	2019-10-26T19:22:23Z	1738073570	2025-01-28T14:10:00Z

5 rows × 211 columns

3) User Story 1: Data Ingestion & "The Clean Up"

Clean the dataset by removing rows with missing or invalid nutritional values and exporting a cleaned dataset for later analysis.

```

# User Story 1: Data Cleaning

# Define the columns required for core analysis and cleaning
required_columns = [
    'product_name',
    'categories_tags',
    'ingredients_text',
    'sugars_100g',
    'proteins_100g',
    'energy_100g'
]

print('Initial dataset shape:', df.shape)
print('Required columns available:', set(required_columns).issubset(df.columns))

# Convert numeric columns to numeric data types and coerce invalid values to NaN
numeric_columns = ['sugars_100g', 'proteins_100g', 'energy_100g']
for col in numeric_columns:

```

```

df[col] = pd.to_numeric(df[col], errors='coerce')

# Drop rows where critical fields are missing or blank
clean_df = df.dropna(subset=['product_name', 'sugars_100g', 'proteins_100g'])
clean_df = clean_df[clean_df['product_name'].astype(str).str.strip() != '']

# Filter out biologically impossible values
clean_df = clean_df[
    (clean_df['sugars_100g'] >= 0) & (clean_df['sugars_100g'] <= 100) &
    (clean_df['proteins_100g'] >= 0) & (clean_df['proteins_100g'] <= 100) &
    (clean_df['energy_100g'] >= 0) & (clean_df['energy_100g'] <= 4000)
]

# Report cleaning results
print('Cleaned dataset shape:', clean_df.shape)
print('Rows removed:', len(df) - len(clean_df))
print('\nMissing value counts in cleaned dataset:')
print(clean_df[['product_name', 'sugars_100g', 'proteins_100g', 'energy_100g']].isna().sum())

# Show sample of cleaned data
display(clean_df.head())

# Export the cleaned dataset for later analysis
clean_data_path = Path('data/cleaned_openfoodfacts.csv.gz') # Changed path to 'data/'
clean_data_path.parent.mkdir(parents=True, exist_ok=True) # Create parent directory if it doesn't exist
clean_df.to_csv(clean_data_path, index=False, compression='gzip')
print('\nCleaned dataset exported to:', clean_data_path.resolve())

```

```

Initial dataset shape: (500000, 211)
Required columns available: True
Cleaned dataset shape: (109514, 211)
Rows removed: 390486

```

```

Missing value counts in cleaned dataset:
product_name      0
sugars_100g       0
proteins_100g     0
energy_100g       0
dtype: int64

```

	code	url	creator	created_t	created_datetime	last_modified_t	last
689	1000	en.openfoodfacts.org/product/0000...	openfoodfacts-contributors	1565750590	2019-08-14T02:43:10Z	1774629663	20
690	100000004	en.openfoodfacts.org/product/0000...	kiliweb	1603360707	2020-10-22T09:58:27Z	1774631951	20
691	100000150	en.openfoodfacts.org/product/0000...	macrofactor	1733481879	2024-12-06T10:44:39Z	1774551293	20
797	100000724	en.openfoodfacts.org/product/0000...	kiliweb	1570364739	2019-10-06T12:25:39Z	1773881458	20
829	100002301	en.openfoodfacts.org/product/0000...	kiliweb	1649189826	2022-04-05T20:17:06Z	1774544741	20

5 rows × 211 columns

Cleaned dataset exported to: /content/data/cleaned_openfoodfacts.csv.gz

✓ 4) User Story 2: The Category Wrangler

Parse `categories_tags`, normalize category strings, and assign each product a primary high-level snack category.

```

# User Story 2: Category Wrangling

# If the cleaned data is not already in memory, load it from the exported cleaned file.
clean_path = Path('data/cleaned_openfoodfacts.csv.gz')
if 'clean_df' not in globals():
    if clean_path.exists():
        clean_df = pd.read_csv(clean_path, compression='gzip', low_memory=False)
        print('Loaded cleaned dataset from:', clean_path.resolve())
    else:
        raise FileNotFoundError(
            'Cleaned dataset not found. Run User Story 1 to create data/cleaned_openfoodfacts.csv.gz'

```

```

)

# Parse category tags into normalized keyword lists
def parse_categories(tag_string):
    if pd.isna(tag_string):
        return []
    tags = [tag.strip().lower() for tag in tag_string.split(',') if tag.strip()]
    normalized = [tag.split(':', 1)[-1] if ':' in tag else tag for tag in tags]
    return normalized

clean_df['category_list'] = clean_df['categories_tags'].apply(parse_categories)

# Primary category mapping rules
category_map = {
    'sweet snacks': ['sweet-snacks', 'candy', 'chocolate', 'cakes', 'cookies', 'biscuits', 'pastries', 'desserts'],
    'salty snacks': ['salted-snacks', 'savory-snacks', 'crisps', 'chips', 'pretzels', 'savory-snacks', ],
    'protein bars': ['protein-bars', 'energy-bars', 'bar', 'bars', 'snack-bars'],
    'breakfast cereals': ['cereals', 'breakfast-cereals', 'granola', 'muesli'],
    'nuts & seeds': ['nuts', 'seeds', 'trail-mixes', 'peanuts', 'almonds', 'cashews'],
    'dairy & alternatives': ['yogurts', 'cheese', 'dairy', 'plant-based', 'milk-based', 'soy-products'],
    'ready-to-eat meals': ['ready-meals', 'sandwiches', 'meal-replacements'],
}

# Assign primary category based on keyword precedence
def assign_primary_category(tags):
    for category, patterns in category_map.items():
        if any(pattern in tag for pattern in patterns) for tag in tags:
            return category
    return 'other snacks'

clean_df['primary_category'] = clean_df['category_list'].apply(assign_primary_category)

# Remove fallback uncategorized products so analysis only uses known snack categories
clean_df = clean_df[clean_df['primary_category'] != 'other snacks'].copy()

# Report category assignment coverage
category_counts = clean_df['primary_category'].value_counts(dropna=False)
print('Primary category distribution:')
print(category_counts)

# Show sample rows for each major category
sample_rows = clean_df.groupby('primary_category').head(2)[['product_name', 'primary_category', 'categories_tags']]
display(sample_rows)

# Save the category-assigned dataset for next steps
category_path = Path('data/cleaned_openfoodfacts_with_categories.csv.gz')
clean_df.to_csv(category_path, index=False, compression='gzip')
print('\nSaved category-assigned dataset to:', category_path.resolve())

```

Primary category distribution:

primary_category	
sweet snacks	12173
breakfast cereals	11473
dairy & alternatives	9236
nuts & seeds	1747
salty snacks	1560
ready-to-eat meals	309
protein bars	286

Name: count, dtype: int64

	product_name	primary_category	categories_tags
865	Eirn original curry Sauce	dairy & alternatives	en:plant-based-foods-and-beverages,en:plant-ba...
875	Donut Milka	sweet snacks	en:snacks,en:sweet-snacks,en:biscuits-and-cake...
890	Véritable pâte à tartiner noisettes chocolat noir	sweet snacks	en:breakfasts,en:spreads,en:sweet-spreads,fr:p...
892	Familjepack Schnitzel Frysta	dairy & alternatives	en:plant-based-foods-and-beverages,en:plant-ba...
1002	Tostadas	breakfast cereals	en:plant-based-foods-and-beverages,en:plant-ba...
1113	Purée de pommes de terre	breakfast cereals	en:plant-based-foods-and-beverages,en:plant-ba...
1590	Formula 1 nutritional shake mix fragola delight	ready-to-eat meals	en:meals,en:dietary-supplements,en:meal-replac...
1620	pumpkin seeds	nuts & seeds	en:plant-based-foods-and-beverages,en:plant-ba...
1701	Dark Chocolate Turbinado Sea Salt Almonds	nuts & seeds	en:plant-based-foods-and-beverages,en:plant-ba...
1810	Barbecue Protein Chips	salty snacks	en:snacks,en:salty-snacks,en:appetizers,en:chi...
1818	Le Jambon de nos Régions	salty snacks	en:snacks,en:salty-snacks,en:appetizers,en:chi...
2682	Complete	ready-to-eat meals	en:meals,en:meal-replacements
7396	55 protein bar	protein bars	en:dietary-supplements,en:bodybuilding-supplem...
7468	75 protein bar	protein bars	en:dietary-supplements,en:bodybuilding-supplem...

Saved category-assigned dataset to: /content/data/cleaned_openfoodfacts_with_categories.csv.gz

Next steps: [Generate code with sample rows](#) [New interactive sheet](#)

5) User Story 3: The "Nutrient Matrix" Visualization

Create an interactive scatter plot comparing Sugar vs Protein across primary categories, plus quadrant counts to identify the underserved market segment.

```
# Load category-assigned data if it is not already present
from pathlib import Path
import pandas as pd
import numpy as np
import plotly.express as px

category_path = Path('data/cleaned_openfoodfacts_with_categories.csv.gz')

if 'clean_df' not in globals() or 'primary_category' not in clean_df.columns:
    if category_path.exists():
        clean_df = pd.read_csv(
            category_path,
            compression='gzip',
            low_memory=False
        )
        print('Loaded category-assigned dataset from:', category_path.resolve())
    else:
        raise FileNotFoundError(
            'Category-assigned dataset not found. '
            'Run User Story 2 to create '
            'data/cleaned_openfoodfacts_with_categories.csv.gz'
        )

# -----
# DATA-DRIVEN THRESHOLDS
# -----
# High Protein = top 25% of products by protein content
# Low Sugar = bottom 25% of products by sugar content
```

```

PROTEIN_THRESHOLD = clean_df['proteins_100g'].quantile(0.75)
SUGAR_THRESHOLD = clean_df['sugars_100g'].quantile(0.25)

print('Nutrient Thresholds Used:')
print(f'High Protein Threshold: >= {PROTEIN_THRESHOLD:.2f} g/100g')
print(f'Low Sugar Threshold: <= {SUGAR_THRESHOLD:.2f} g/100g')

# -----
# QUADRANT CLASSIFICATION
# -----

conditions = [
    (
        (clean_df['proteins_100g'] >= PROTEIN_THRESHOLD) &
        (clean_df['sugars_100g'] <= SUGAR_THRESHOLD)
    ),
    (
        (clean_df['proteins_100g'] >= PROTEIN_THRESHOLD) &
        (clean_df['sugars_100g'] > SUGAR_THRESHOLD)
    ),
    (
        (clean_df['proteins_100g'] < PROTEIN_THRESHOLD) &
        (clean_df['sugars_100g'] <= SUGAR_THRESHOLD)
    )
]

choices = [
    'High Protein / Low Sugar',
    'High Protein / High Sugar',
    'Low Protein / Low Sugar'
]

clean_df['quadrant'] = np.select(
    conditions,
    choices,
    default='Low Protein / High Sugar'
)

# -----
# QUADRANT SUMMARY
# -----

quadrant_order = [
    'High Protein / Low Sugar',
    'High Protein / High Sugar',
    'Low Protein / Low Sugar',
    'Low Protein / High Sugar'
]

quadrant_counts = (
    clean_df['quadrant']
    .value_counts()
    .reindex(quadrant_order, fill_value=0)
)

print('\nQuadrant Counts:')
print(quadrant_counts)

# Identify underserved quadrant
underserved = quadrant_counts.idxmin()

print(
    f'\nMost Underserved Quadrant: '
    f'{underserved} '
    f'({quadrant_counts[underserved]:,} products)'
)

# -----
# SCATTER PLOT
# -----

fig = px.scatter(
    clean_df,
    x='sugars_100g',
    y='proteins_100g',

```

```

        color='primary_category',
        hover_data=[
            'product_name',
            'primary_category',
            'sugars_100g',
            'proteins_100g',
            'quadrant'
        ],
        title=(
            'Nutrient Matrix: Protein vs Sugar by Primary Category<br>'
            f'<sup>'
            f'High Protein ≥ {PROTEIN_THRESHOLD:.1f}g | '
            f'Low Sugar ≤ {SUGAR_THRESHOLD:.1f}g'
            f'</sup>'
        ),
        width=1100,
        height=700,
        opacity=0.7
    )

    fig.update_layout(
        xaxis_title='Sugars (g per 100g)',
        yaxis_title='Proteins (g per 100g)',
        legend_title='Primary Category',
        template='plotly_white'
    )

    # Vertical threshold line (Sugar)
    fig.add_shape(
        type='line',
        x0=SUGAR_THRESHOLD,
        x1=SUGAR_THRESHOLD,
        y0=0,
        y1=clean_df['proteins_100g'].max(),
        line=dict(color='black', dash='dash')
    )

    # Horizontal threshold line (Protein)
    fig.add_shape(
        type='line',
        x0=0,
        x1=clean_df['sugars_100g'].max(),
        y0=PROTEIN_THRESHOLD,
        y1=PROTEIN_THRESHOLD,
        line=dict(color='black', dash='dash')
    )

    # Quadrant annotations
    fig.add_annotation(
        x=SUGAR_THRESHOLD * 0.5,
        y=clean_df['proteins_100g'].max() * 0.92,
        text='High Protein / Low Sugar',
        showarrow=False
    )

    fig.add_annotation(
        x=clean_df['sugars_100g'].max() * 0.75,
        y=clean_df['proteins_100g'].max() * 0.92,
        text='High Protein / High Sugar',
        showarrow=False
    )

    fig.add_annotation(
        x=SUGAR_THRESHOLD * 0.5,
        y=PROTEIN_THRESHOLD * 0.5,
        text='Low Protein / Low Sugar',
        showarrow=False
    )

    fig.add_annotation(
        x=clean_df['sugars_100g'].max() * 0.75,
        y=PROTEIN_THRESHOLD * 0.5,
        text='Low Protein / High Sugar',
        showarrow=False
    )

    fig.show()

```

```
# -----
# EXPORT QUADRANT SUMMARY
# -----

summary_path = Path('data/quadrant_summary.csv')

(
    quadrant_counts
    .reset_index()
    .rename(columns={
        'index': 'Quadrant',
        'quadrant': 'Count'
    })
    .to_csv(summary_path, index=False)
)

print('\nSaved quadrant summary to:', summary_path.resolve())

# -----
# BUSINESS INSIGHT
# -----

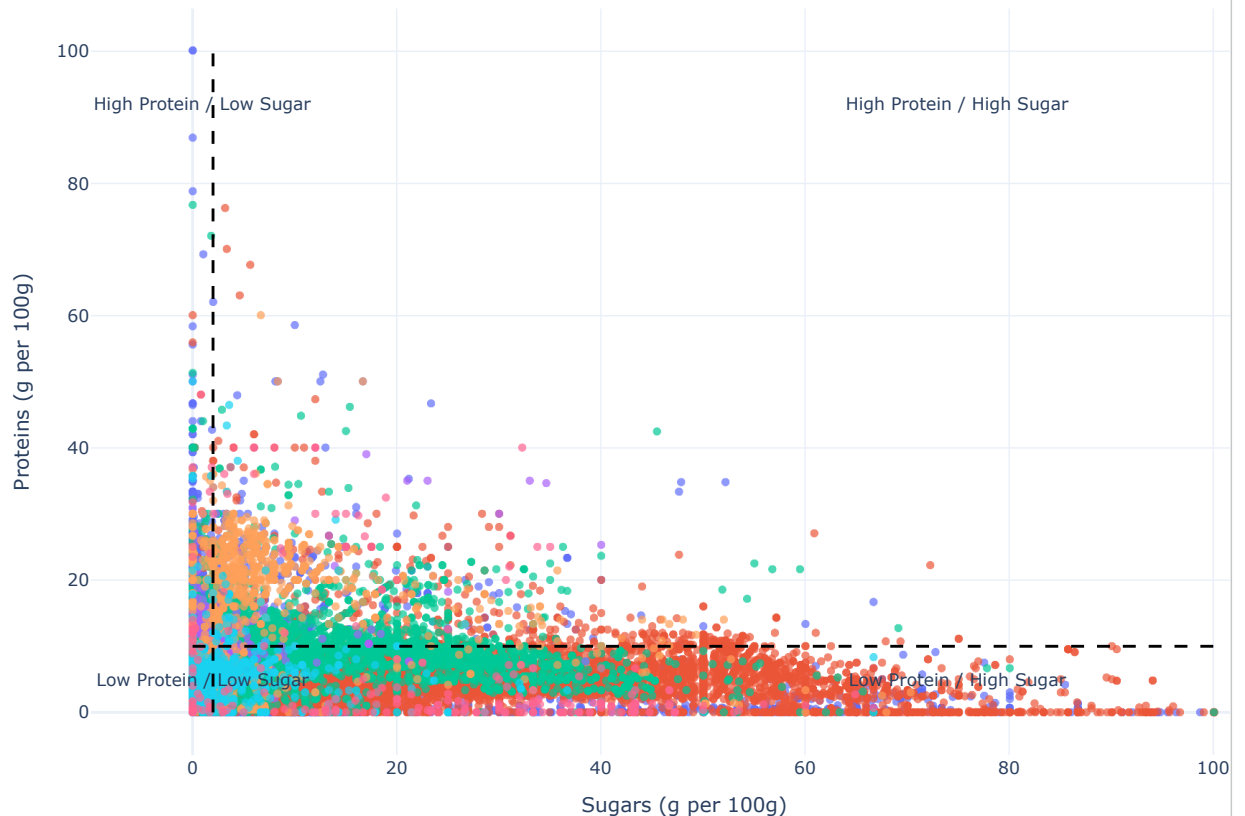
print('\nBusiness Insight:')
print(
    f'The biggest market opportunity appears to be in the '
    f'"{underserved}" quadrant, suggesting relatively few '
    f'products currently offer this nutrient profile.'
)
```

Nutrient Thresholds Used:
High Protein Threshold: ≥ 10.00 g/100g
Low Sugar Threshold: ≤ 2.00 g/100g

Quadrant Counts:
quadrant
High Protein / Low Sugar 3241
High Protein / High Sugar 6155
Low Protein / Low Sugar 6233
Low Protein / High Sugar 21155
Name: count, dtype: int64

Most Underserved Quadrant: High Protein / Low Sugar (3,241 products)

Nutrient Matrix: Protein vs Sugar by Primary Category
High Protein ≥ 10.0 g | Low Sugar ≤ 2.0 g



Saved quadrant summary to: /content/data/quadrant_summary.csv

Business Insight:
The biggest market opportunity appears to be in the "High Protein / Low Sugar" quadrant, suggesting relatively few

6) User Story 4: The Recommendation

Create a clear, data-backed product recommendation for the most underserved quadrant.

```
# User Story 4: Recommendation

# Reuse the quantile-based thresholds from Story 3 if available,
# otherwise recompute them the same way Story 3 does.
if 'PROTEIN_THRESHOLD' not in globals() or 'SUGAR_THRESHOLD' not in globals():
    PROTEIN_THRESHOLD = clean_df['proteins_100g'].quantile(0.75)
    SUGAR_THRESHOLD = clean_df['sugars_100g'].quantile(0.25)

# Identify the underserved quadrant from Story 3
quadrant_counts = clean_df['quadrant'].value_counts().reindex([
    'High Protein / Low Sugar',
    'High Protein / High Sugar',
    'Low Protein / Low Sugar',
    'Low Protein / High Sugar'
])
```

```

], fill_value=0)

underserved_quadrant = quadrant_counts.idxmin()
underserved_count = int(quadrant_counts.loc[underserved_quadrant])

quadrant_subset = clean_df[clean_df['quadrant'] == underserved_quadrant]
category_counts_underserved = quadrant_subset['primary_category'].value_counts()

if not category_counts_underserved.empty:
    recommended_category = category_counts_underserved.idxmax()
    recommended_category_count = int(category_counts_underserved.max())
else:
    recommended_category = 'Unknown'
    recommended_category_count = 0

recommendation = (
    f"Based on the data, the biggest market opportunity is in {recommended_category}, "
    f"specifically targeting products with at least {PROTEIN_THRESHOLD:.0f}g of protein "
    f"and no more than {SUGAR_THRESHOLD:.0f}g of sugar per 100g. "
    f"Currently there are only {underserved_count:,} products in the {underserved_quadrant} quadrant "
    f"and only {recommended_category_count:,} of those are in the {recommended_category} category."
)

print("=== Recommendation ===")
print(recommendation)

# Save recommendation summary
recommendation_path = Path('data/recommendation_summary.txt')
recommendation_path.write_text(recommendation)
print("\nSaved recommendation to:", recommendation_path.resolve())

```

```

=== Recommendation ===
Based on the data, the biggest market opportunity is in dairy & alternatives, specifically targeting products with

Saved recommendation to: /content/data/recommendation_summary.txt

```

✓ 7) User Story 5: The Hidden Gem

Analyze the ingredients of high-protein products to extract the top protein sources used in the opportunity cluster.

```

import re
from collections import Counter

high_protein_subset = clean_df[clean_df['protein_level'] == 'High Protein']

protein_source_keywords = [
    'whey', 'soy', 'peanut', 'almond', 'egg', 'milk', 'casein', 'pea',
    'rice', 'hemp', 'collagen', 'chicken', 'beef', 'pork', 'fish', 'salmon',
    'shellfish', 'lentil', 'chickpea', 'beans', 'quinoa', 'oat', 'yogurt',
    'cottage cheese', 'cheese', 'coconut', 'turkey', 'duck', 'seitan', 'tofu',
    'tempeh', 'pumpkin seed', 'sesame', 'sunflower', 'chia', 'flax'
]

protein_source_pattern = re.compile(r"\b(" + "|".join(re.escape(src) for src in protein_source_keywords) + r")\b",
                                     flags=re.IGNORECASE)

source_counts = Counter()

for ingredients in high_protein_subset['ingredients_text'].dropna():
    matches = protein_source_pattern.findall(ingredients)
    normalized = [match.lower() for match in matches]
    source_counts.update(normalized)

source_summary = pd.DataFrame(
    source_counts.most_common(), columns=['protein_source', 'count']
)

if source_summary.empty:
    print('No protein source matches were found in the high-protein product ingredients.')
else:
    top_sources = source_summary.head(3)
    print('Top 3 protein sources in the high-protein cluster:')
    for idx, row in top_sources.iterrows():
        print(f"{idx + 1}. {row['protein_source'].title()} ({row['count']:,} products)")

# Save source summary

```

```
# Save source summary
protein_source_path = Path('data/protein_source_summary.csv')
source_summary.to_csv(protein_source_path, index=False)
print('\nSaved protein source summary to:', protein_source_path.resolve())
```

Top 3 protein sources in the high-protein cluster:

1. Milk (2,923 products)
2. Cheese (1,734 products)
3. Soy (1,680 products)

Saved protein source summary to: /content/data/protein_source_summary.csv

✓ Candidate Challenge: Category Market Gap Explorer

A category-level opportunity ranking that uses the underserved nutrient quadrant and average nutrient gaps to identify the most promising snack categories.

```
# Candidate Challenge: Category Market Gap Explorer
if 'quadrant' not in clean_df.columns:
    raise ValueError('Run User Story 3 first so quadrant labels are available.')

if 'underserved' not in globals():
    quadrant_counts = clean_df['quadrant'].value_counts().reindex([
        'High Protein / Low Sugar',
        'High Protein / High Sugar',
        'Low Protein / Low Sugar',
        'Low Protein / High Sugar'
    ], fill_value=0)
    underserved = quadrant_counts.idxmin()

category_gap = (
    clean_df
    .groupby('primary_category')
    .agg(
        total_products=('product_name', 'size'),
        underserved_products=('quadrant', lambda x: (x == underserved).sum()),
        avg_protein=('proteins_100g', 'mean'),
        avg_sugar=('sugars_100g', 'mean'),
    )
    .reset_index()
```

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.